Assignment No. 3
EECS 348
Software Engineering I
Antonio DePaolis
Due: 11:59 PM, Thursday, October 1, 2026

Analysis

1. AI used
    a. CoPilot & Gemini
2. How I accessed AI
    a. I accessed Copilot by going onto the copilot website. I accessed Gemini by going to the Google Gemini website
3. Prompt

Using C++ generate code that will take in a file name from the user which holds a list of commands. Use a MaxHeap to implement a priority queue. The program will prioritize emails for a busy company CEO. This priority queue will shift emails towards the front of the queue based on priority status. The MaxHeap must be implemented using list-based implementation then use it to handle all email prioritizing for the CEO. Create the functions from scratch DO NOT include pre-existing heap modules. Do not forget to include edge cases. Here is the Test File format for the commands

EMAIL <sender category>, <subject line>, <date>

Emails in CEO's inbox should be placed in the queue based on sender category and date. Here is the priority. Boss – read first, Subordinate – read next, Peer – read next, ImportantPerson – read next, OtherPerson – read last

If more than one from a sender then the newest should be read first

EMAIL is followed by space. The rest of the fields are delimited.

Assume <sender category> is one of the five strings listed above.

Assume <subject line> is a string which may contain spaces, but not commas

Assume <date> is in the format: MM-DD-YYY

NEXT

Next email for the CEO to read. Display the information on the terminal in the following format:

Next email:

Sender:

Subject:

Date:

READ

CEO has read email and dealt with it

COUNT

Display current count of unread emails by saying how many how many are to be read

4. Generated code
    a. CoPilot

```cpp
#include <iostream>
#include <fstream>
#include <vector>
#include <sstream>
#include <string>

using namespace std;

struct Email {
    string senderCategory;
    string subject;
    string date;
};

/*
    Convert category to numeric priority.
    Higher number = higher priority.
*/
int getPriority(string category) {
    if (category == "Boss") return 5;
    if (category == "Subordinate") return 4;
    if (category == "Peer") return 3;
    if (category == "ImportantPerson") return 2;
    return 1; // OtherPerson
}

/*
    Converts MM-DD-YYYY into YYYYMMDD integer
    so dates can be compared easily.
*/
int dateToInt(string date) {
    string month = date.substr(0, 2);
    string day = date.substr(3, 2);
    string year = date.substr(6, 4);

    return stoi(year + month + day);
}
```

```
class MaxHeap {
private:
    vector<Email> heap;

    bool higherPriority(const Email& a, const Email& b) {
        int priA = getPriority(a.senderCategory);
        int priB = getPriority(b.senderCategory);

        if (priA != priB)
            return priA > priB;

        return dateToInt(a.date) > dateToInt(b.date);
    }

    void heapifyUp(int index) {
        while (index > 0) {
            int parent = (index - 1) / 2;

            if (higherPriority(heap[index], heap[parent])) {
                swap(heap[index], heap[parent]);
                index = parent;
            }
            else {
                break;
            }
        }
    }

    void heapifyDown(int index) {
        int size = heap.size();

        while (true) {
            int left = 2 * index + 1;
            int right = 2 * index + 2;
            int largest = index;

            if (left < size &&
                higherPriority(heap[left], heap[largest])) {
```

```
                largest = left;
            }

            if (right < size &&
                higherPriority(heap[right], heap[largest])) {
                largest = right;
            }

            if (largest != index) {
                swap(heap[index], heap[largest]);
                index = largest;
            }
            else {
                break;
            }
        }
    }

public:

    void insert(const Email& email) {
        heap.push_back(email);
        heapifyUp(heap.size() - 1);
    }

    Email getMax() {
        if (heap.empty())
            throw runtime_error("Heap is empty.");

        return heap[0];
    }

    void removeMax() {
        if (heap.empty()) {
            cout << "No emails to read." << endl;
            return;
        }

        heap[0] = heap.back();
```

```
        heap.pop_back();

        if (!heap.empty())
            heapifyDown(0);
    }

    int count() const {
        return heap.size();
    }

    bool empty() const {
        return heap.empty();
    }
};

/*
    Removes leading and trailing spaces.
*/
string trim(const string& str) {
    size_t first = str.find_first_not_of(" ");
    size_t last = str.find_last_not_of(" ");

    if (first == string::npos)
        return "";

    return str.substr(first, last - first + 1);
}

int main() {

    string filename;

    cout << "Enter command file name: ";
    getline(cin, filename);

    ifstream infile(filename);

    if (!infile) {
        cout << "Error: Could not open file." << endl;
```

```
        return 1;
    }

    MaxHeap emailQueue;

    string line;

    while (getline(infile, line)) {

        if (line.empty())
            continue;

        if (line.substr(0, 5) == "EMAIL") {

            string data = line.substr(6);

            string sender;
            string subject;
            string date;

            stringstream ss(data);

            getline(ss, sender, ',');
            getline(ss, subject, ',');
            getline(ss, date);

            sender = trim(sender);
            subject = trim(subject);
            date = trim(date);

            Email email;
            email.senderCategory = sender;
            email.subject = subject;
            email.date = date;

            emailQueue.insert(email);
        }

        else if (line == "NEXT") {
```

```
        if (emailQueue.empty()) {
            cout << "\nNo unread emails.\n" << endl;
        }
        else {

            Email next = emailQueue.getMax();

            cout << "\nNext email:" << endl;
            cout << "Sender: " << next.senderCategory << endl;
            cout << "Subject: " << next.subject << endl;
            cout << "Date: " << next.date << endl;
        }
    }

    else if (line == "READ") {

        if (emailQueue.empty()) {
            cout << "No emails available to remove." << endl;
        }
        else {

            Email email = emailQueue.getMax();

            cout << "\nReading email:" << endl;
            cout << "Sender: " << email.senderCategory << endl;
            cout << "Subject: " << email.subject << endl;
            cout << "Date: " << email.date << endl;

            emailQueue.removeMax();
        }
    }

    else if (line == "COUNT") {

        cout << "Unread emails: "
             << emailQueue.count()
             << endl;
    }
```

```
        else {
            cout << "Invalid command: "
                 << line << endl;
        }
    }

    infile.close();

    return 0;
}
```

b. Gemini

```cpp
#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
#include <vector>

// Structure to store Email data
struct Email {
    std::string category;
    std::string subject;
    std::string date; // Format: MM-DD-YYYY
    int priorityScore;
    int year, month, day;

    Email() : priorityScore(0), year(0), month(0), day(0) {}

    Email(const std::string& cat, const std::string& subj, const
std::string& d)
        : category(cat), subject(subj), date(d) {

        // Assign priority weight (Higher value = higher priority)
        if (category == "Boss") priorityScore = 5;
        else if (category == "Subordinate") priorityScore = 4;
        else if (category == "Peer") priorityScore = 3;
        else if (category == "ImportantPerson") priorityScore = 2;
        else priorityScore = 1; // "OtherPerson" or any default edge
case

        // Parse date string (MM-DD-YYYY) for exact comparison
        std::stringstream ss(date);
        char dash1, dash2;
        ss >> month >> dash1 >> day >> dash2 >> year;
    }

    // Returns true if 'this' email has HIGHER priority than 'other'
    bool isHigherPriorityThan(const Email& other) const {
        if (this->priorityScore != other.priorityScore) {
            return this->priorityScore > other.priorityScore;
```

```
        }
        // If category is the same, compare dates (Newest first)
        if (this->year != other.year) return this->year > other.year;
        if (this->month != other.month) return this->month >
other.month;
        return this->day > other.day;
    }
};

// Custom List-based MaxHeap implementation
class MaxHeap {
private:
    std::vector<Email> heap;

    void heapifyUp(int index) {
        while (index > 0) {
            int parentIndex = (index - 1) / 2;
            if (heap[index].isHigherPriorityThan(heap[parentIndex])) {
                std::swap(heap[index], heap[parentIndex]);
                index = parentIndex;
            } else {
                break;
            }
        }
    }

    void heapifyDown(int index) {
        int size = static_cast<int>(heap.size());
        while (index < size) {
            int leftChild = 2 * index + 1;
            int rightChild = 2 * index + 2;
            int largest = index;

            if (leftChild < size &&
heap[leftChild].isHigherPriorityThan(heap[largest])) {
                largest = leftChild;
            }
            if (rightChild < size &&
heap[rightChild].isHigherPriorityThan(heap[largest])) {
```

```
                largest = rightChild;
            }

            if (largest != index) {
                std::swap(heap[index], heap[largest]);
                index = largest;
            } else {
                break;
            }
        }
    }

public:
    void push(const Email& email) {
        heap.push_back(email);
        heapifyUp(static_cast<int>(heap.size()) - 1);
    }

    // Edge Case: Handling empty heap access safely
    bool peek(Email& topEmail) const {
        if (heap.empty()) return false;
        topEmail = heap[0];
        return true;
    }

    bool pop() {
        if (heap.empty()) return false;
        heap[0] = heap.back();
        heap.pop_back();
        heapifyDown(0);
        return true;
    }

    int size() const {
        return static_cast<int>(heap.size());
    }

    bool empty() const {
        return heap.empty();
```

```
    }
};

// Helper function to trim whitespace from strings
std::string trim(const std::string& str) {
    size_t first = str.find_first_not_of(" \t\r\n");
    if (first == std::string::npos) return "";
    size_t last = str.find_last_not_of(" \t\r\n");
    return str.substr(first, (last - first + 1));
}

int main() {
    std::string filename;
    std::cout << "Enter command file name: ";
    std::cin >> filename;

    std::ifstream infile(filename);

    // Edge Case: File failure opening
    if (!infile.is_open()) {
        std::cerr << "Error: Unable to open file '" << filename << "'."
<< std::endl;
        return 1;
    }

    MaxHeap priorityQueue;
    std::string line;

    while (std::getline(infile, line)) {
        line = trim(line);
        if (line.empty()) continue; // Skip empty lines

        if (line.rfind("EMAIL", 0) == 0) {
            // Processing EMAIL command
            std::string data = line.substr(5); // Remove "EMAIL"
            std::stringstream ss(data);
            std::string category, subject, date;

            if (std::getline(ss, category, ',') &&
```

```
                std::getline(ss, subject, ',') &&
                std::getline(ss, date, ',')) {

                category = trim(category);
                subject = trim(subject);
                date = trim(date);

                Email mail(category, subject, date);
                priorityQueue.push(mail);
            }
        }
        else if (line == "NEXT") {
            Email nextEmail;
            // Edge Case: CEO requests NEXT when inbox is empty
            if (priorityQueue.peek(nextEmail)) {
                std::cout << "Next email:\n";
                std::cout << "Sender: " << nextEmail.category << "\n";
                std::cout << "Subject: " << nextEmail.subject << "\n";
                std::cout << "Date: " << nextEmail.date << "\n\n";
            } else {
                std::cout << "No emails in queue.\n\n";
            }
        }
        else if (line == "READ") {
            // Edge Case: Processing READ when inbox is empty
            if (!priorityQueue.pop()) {
                std::cout << "No emails to read.\n\n";
            }
        }
        else if (line == "COUNT") {
            std::cout << "There are " << priorityQueue.size() << "
emails to be read.\n\n";
        }
    }

    infile.close();
    return 0;
}
```

5. Analyze the code in terms of
    a. Correctness

Both code were able to be ran but Copilot added a print for the READ command that prints off what NEXT prints but changes it to Reading email:

Edgecases

- Copilot
    - When given an unknown command it states invalid command and stops running
    - When given an EMAIL command without all the parameters it breaks
- Gemini
    - When given an unknown command it skips over it
    - If EMAIL doesn't have all parameters it skips over it.
    - When given NEXT and READ with no emails left prints out a statement stating no emails in queue or no emails to read

    b. execution time

- Time Complexity
    - Since they use the same systems their time complexity in theory is the same.
- Execution Time
    - Though the execution time is different. In Copilot code when comparing priority it uses a function higherPriority that calls another function dateToInt to compare its date that is stored within a string.
    - While in Gemini it stores the date into three integers along with priority score integers when first created so it simply just compares the four integers when checking priority.
    - This results in Copilot taking more time than Gemini which isn't as noticeable with a small amount of emails but with a lot it can be visible.

    c. space complexity

- Space Complexity
    - Both have similar space complexity as they store an email of size n for a space complexity of O(n)
    - The problem occurs during comparison as for Copilot it creates a copy for each email every time it makes a comparison during heapifyUp or heapifyDown. This results in it taking up more RAM during the loop which can cause issues

d. maintainability as explained in the “GenAI Code Comparison” lecture.

1. Comments
    a. Copilot
        i. Has three comments with two describing two global functions and one describing a string trimmer that removes whitespace
    b. Gemini
        i. Has more comments than Copilot but not enough to describe every function
2. Readability
    a. Both
        i. Both create a structure for Email as well as a class for Maxheap
        ii.
    b. Copilot
        i. Has functions outside of MaxHeap used only by MaxHeap to compare email structures
    c. Gemini
        i. Has the comparison done during the creation of the Email structure keeping it all in one place
        ii.
3. Edge case/ Error handling
    a. Copilot
        i. Has almost no edge cases resulting in the code breaking if there is an empty line or just if a command isn’t fully filled out
        ii. If there is a space rather than just a newline after a command the program will just state its invalid as it doesn’t remove that whitespace at the end of the command
        iii. If commands are called while empty prints a statement stating it can’t remove an email or for NEXT that there are no unread emails.
    b. Gemini
        i. Can handle empty lines and skip them
        ii. Can handle invalid commands and skips them
        iii. If commands are called while empty prints a statement stating it can’t read or for NEXT that there are no emails in queue.

6. Select one to use as the basis for this assignment and justify why you picked it.
    a. I decided to choose the Gemini code as there are just less errors within the code itself. It is easier to read. Has more edge cases, less lines to work with. Copilot on the other hand has a few edge cases, doesn’t remove whitespace for other commands, is not an effective priority handler, and has more lines than needed.

7. Explain what you did to improve it in terms of correctness, execution time, space complexity, and maintainability.

For correctness I will add a isValidDate function to make sure inputted date is valid and not impossible. For execution time and space complexity I will not make any changes to them. For maintainability I will comment on every line.